Characterizing the Cornerstone Octree Construction Method

Henry Murdock, William Halpin, Rayan Singh
University of Illinois at Urbana-Champaign
{henrygm3,lhalpin2,rayans2}@illinois.edu

Abstract

An octree is a structure which partitions a 3D space into octants recursively based on positions of objects within the domain. This structure is heavily utilized in scientific simulation, computer graphics, and robotics. One major use case of an octree is in the field of particle simulations. Simulations calculate the particle interactions between millions, billions, or even trillions of particles, and this complexity of calculating per-pair interactions becomes prohibitive for the system.

Recent works have expanded upon traditional octree construction methods to allow for distributed simulation on graphics processing units (GPUs). We explore and benchmark a novel method called Cornerstone [7] that implements these methods and develops a construction method that allows an octree to be rebalanced on-the-fly which is favorable for particle simulations or any problem which frequently perturbs the positions of the points in some way. This optimization is particularly important in the distributed construction, where it reduces the communication cost.

We discover that for cases where we have a cluster node of GPUs, the communication cost is negligible, and the distributed construction is instead dominated by the calculation of peer ranks. We study the effect of different algorithm parameters on performance. We optimize the local construction method and achieve performance improvements up to 2.45× for 100 million particles.

1 Introduction

Scientific simulation, computer graphics, collision detection, and many other fields utilize data structures called octrees. An octree allows a three dimensional space to be broken down hierarchically into a tree structure with up to eight children per node. Meshless point clouds, such as a group of particles, are an important application of such structures, especially when paired with Barnes-Hut or Fast Multipole Methods which allow reductions in the runtime of force calculations between points from $O(n^2)$ to $O(n \log(n))$ and roughly $O(n)$ respectively. This has driven the development of octree construction methods that operate on GPUs and across multiple nodes in clusters and supercomputers.

One of the most interesting new methods for meshless point clouds is Cornerstone [7]. In the case of simulating the movement of particles in dynamic systems, the octree must be recalculated at each time step, however, the primary advantage of Cornerstone is to rebalance the tree rather than rebuild from scratch. Cornerstone has been heavily used on exascale supercomputers, however not all problems which may benefit from this method require such systems. Furthermore, a profile of the code internals and how the method performs across varying conditions of particle distributions, parameters, and other variants does not appear publicly but may help to optimize and evidence the method's usefulness across varying systems and problem spaces.

This project seeks to do such a characterization and will focus closely on the mapping of Cornerstone to the current GPU architectures examining any kernel level metrics of interest, memory movement at the intra and inter node scope including caches, as well as the interactions of communication and computation cost with different algorithm parameters.

2 Cornerstone

To characterize Cornerstone's performance, we first need to understand the algorithm. Note that there are two methods of interest, the method where an octree is constructed locally on a node, and the case where a locally essential tree (LET) is used for distributed simulation. We review the local case first and then advance to the distributed case as an extension. We focus primarily on the code needed and how it maps to GPU architectures, for further insight into the algorithm reference Cornerstone [7]. An outline of the construction steps is given for the local case.

- Compute a space filling curve (Morton or Hilbert)
- Sort the keys and rearrange any properties associated with the key such as the position, mass, charge, or any variable associated with a particle of interest during the computation using octrees.
- Update the leaf nodes
- Update the internal tree structure

Additionally, Cornerstone has 6 critical parameters which are tabulated below. These will also be important parameters for characterization.

2.1 Space Filling Curves

Informally, a space filling curve will take points existing in space and map them to integer keys. When the original particles are sorted based on these keys and one traverses the path through them the resulting curve will recursively move through the domain octant by octant (in the case of

Table 1. Cornerstone Parameters

Parameter	Description
Precision	FP64 or FP32
θ	The MAC parameter
N_{crit}	Maximal # of points in a leaf
h	Smoothing parameter
L	The maximal depth of the octree
SFC	Morton or Hilbert

three dimensional points). In this way, a space filling curve is a kind of fractal.

We are interested in two types of space filling curves namely Morton Z-order curves and Hilbert curves. The path of a Morton Z-order curve may be viewed in figure 1 and shows the distinct z-pattern. Morton codes are computationally efficient to calculate, requiring just a few bit shifts, but as is also evident visually in figure 1 the curve may "jump" octants. This may result in unfavorable communication patterns between nodes.

The Hilbert curve on the other hand takes on a compact U shape as seen in figure 2. Hilbert codes offer a "compact" mapping which may reduce the cost of communication but come at the cost of increased computational inefficiency. In both cases the calculation is an elementwise GPU kernel.

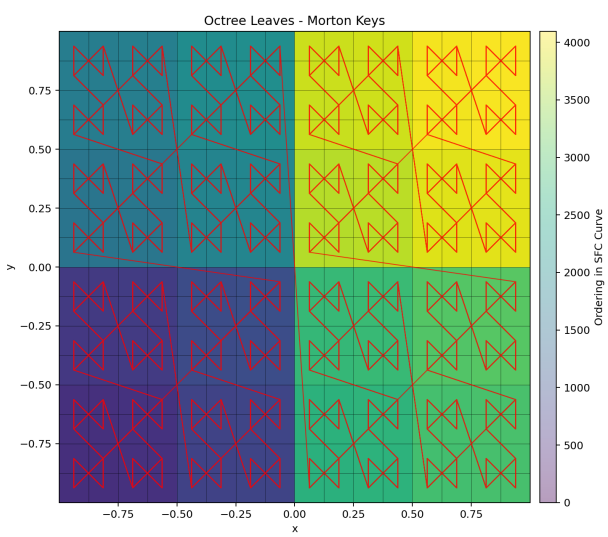

Figure 1. The leaves of an octree for a uniform distribution on the unit cube, colored with respect to their Morton Z-order curve. The red trace plots the curves path through the leaves. Note the jump between different octants which causes issues in communication.

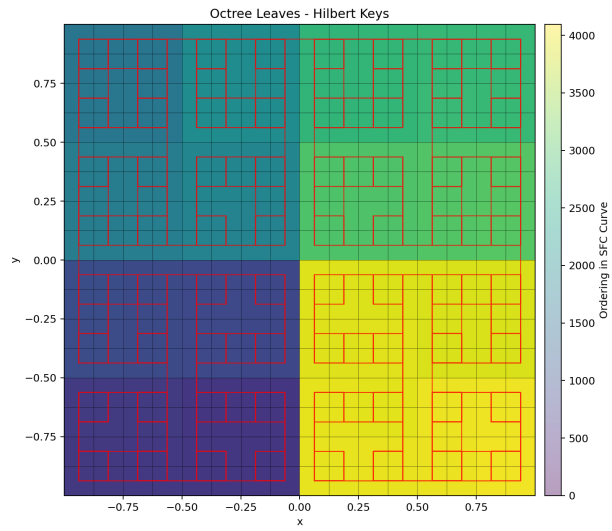

Figure 2. The leaves of an octree for a uniform distribution on the unit cube, colored with respect to their Hilbert curve. The red trace plots the Hilbert curve path through the leaves. Note that the jump that occurs in 1 disappears leading to a compact domain.

2.2 Radix Sorting and Gathers

We will not detail the Radix sort and Gather implementations as they are well known. The reason these operations are necessary is to sort the keys and produce the in-order arrangement of keys according to the space filling curve that recursively partitions the space into octants. The keys will have some attributes including x, y, z but may have others based on the application. These also need to be aligned with the sorted keys. In Cornerstone these are distinct kernels with a Radix sort and k gathers for the number of properties to be sorted. The radix sort is done using the CUB libraries implementation DeviceRadixSort::SortPairs. The gathering of particles in the single node case is left to the user but we used the Thrust library's gather implementation which has competitive performance.

2.3 Update Leaf Nodes

The namesake of Cornerstone [7] is the Cornerstone array called K. K is a sorted array of SFC keys of length $n_l + 1$, n_l being the number of leaves, with the first element being 0 and the last element being 8^L, with the difference between one element and its successor being 8^l, where $l \leq L$. Then K will define a unique octree but represents explicitly the leaves. Cornerstone [7] also uses an N_{crit} parameter which defines the maximum allowable number of particles a leaf node may have. In other words, this parameter defines the resolution of the octree or depth at which it must be resolved.

Then the steps to generate the Cornerstone array from the particle SFCs are to calculate a histogram of the particle SFCs belonging to each leaf node and then perform three steps:

1. Compute the rebalancing decision for each node, a code where 8 indicates $N_{crit} < N_i$ and so the node should be subdivided, 0 indicates the node should be replaced, and 1 indicates the leaf should be unchanged.
2. Perform an exclusive prefix sum of the array of computed rebalancing decision codes, this leads to each element in the result being the new index of the node in the corresponding new Cornerstone array. The final element is the number of nodes in the new Cornerstone array.
3. Finally, if the rebalancing decision for node i is 1 then the key is copied to the new index calculated in the exclusive prefix sum. If the decision is to split, the code is 8 and we copy the keys of the children of the original node to the new range.

A Cornerstone array is then balanced once no node needs to be split or merged, which means that every leaf node has a particle count $\leq N_{crit}$ and every internal node has a particle count $> N_{crit}$. By iteratively applying these steps until we reach the criteria for a balanced Cornerstone array we define a unique balanced octree.

Regarding performance, note that each kernel launch here must occur in serial, no stream parallelism is possible as global synchronization is necessary between each step. The calculation of the rebalancing decision is an elementwise kernel and in practice may use larger multiples of 8 to indicate multiple levels need to be merged. Additionally, a memory allocation is necessary and has to read the value of the last element of the exclusive prefix sum to calculate the number of new leaf nodes.

2.4 Update Internal Nodes

Finally, the internal tree structure must be created allowing us to traverse the tree and ultimately we are left with three arrays of integers, note $n = n_i + n_l$ with n_i being the number of internal nodes and n_l being the number of leaf nodes:

1. NK of size n, with each value being within the range defined by the SFC size supported by L which is $[0, 2^{3L})$. This is the SFC key for each node both leaves and internal nodes, note that because the SFC can decode to 3D points this also tells us the 3D positions of each node.
2. CO of size n, with each value being in the range $[1, n)$. This is the connectivity array.
3. LO of size $L+2$ with each value being in the range $[1, n)$ which are the tree level offsets, that is the j element of LO gives the index into CO and NK that is the first node in the level j of the tree.

This requires several operations, the first is allocating NK and copying the Cornerstone array to the portion of the range for leaf nodes, this is an elementwise kernel. Then the algorithm computes another elementwise kernel that does an xor of two adjacent keys in the Cornerstone array and then counts the leading zeros. Then NK is sorted via radix sort which gives a breadth-first node ordering, again using the CUB implementations. Then the level offsets LO are generated with a binary search of NK, another elementwise kernel. And finally the connectivity array CO is generated by doing a binary search for the first child of a node using the level offsets to reduce the problem size.

An important performance note here that will show up in 4 is that the computation of SFC keys and the subsequent radix sort and gathers all operate on n_p which is the number of bodies in simulation space which is much larger than the computation being done in the later two steps where we operate only on the leaf nodes after the histogram of the particles SFC codes, assuming N_{crit} is sufficiently large (if $N_{crit} = 1$ then the leaves are the particles SFC codes themselves). We discuss this in further detail in 4.

2.5 Domain Decomposition

The above process outlines the process to build a balanced octree on a single rank. This method can be extended, however, to allow multiple peers to decompose the domain among themselves, this allows a rank to maintain a well resolved local domain with a coarser global domain. This modifies the process of generating the octree to compute a global octree and a focused Locally Essential Tree.

The global octree allows us to decompose the domain of particles among the peer ranks based on the SFC, calculating the coarse global domain with a higher N_{crit} called $N_{crit,glob}$. This process modifies the above algorithm so that the initial histogram step in the Cornerstone array rebalance does a global sum of the histograms from each rank via *MPI_Allreduce*. Then performing the balancing steps using the histogram of the global domain rather than the local domain yields a balanced global octree. Such an algorithm would incur unacceptable communication costs at high resolution (lower N_{crit}) as it involves an all-to-all communication which would be iterated on; more resolved domains requiring more iterations to balance the tree. However at a coarser resolution we can use this to efficiently decompose the global domain by partitioning the Cornerstone array of the global domain among the ranks. The collective communication of this method is $O(nlogn)$ where n is the total number of particles across all ranks.

2.6 Locally Essential Trees

Once the space filling curve is decomposed among n_{ranks} subdomains the algorithm attempts to find a Locally Essential Tree which includes the nodes necessary for computation within the local subdomain F. Here Cornerstone [7] incorporates physical context into the algorithm considering both short-range forces and long-range forces. Both require a high

resolution within F and at the exterior surface of F for halo particle discovery. Long-range forces are assumed to be computed via Barnes-Hut or the Fast Multipole Method both of which use a Multipole Acceptance Criterion (MAC) function to determine whether the multipole approximation applies given a source node and target point or node; the multipole approximation being that the force can be computed by using the source node as a kind of super particle that acts on the target with a force relative to the number of particles within it. The most common MAC is $r_{min} > l/\theta$, where r_{min} is the minimum distance between source and target, l is the source node edge length, and θ is a controllable accuracy parameter as noted in the table above. If this inequality is true then the multipole approximation is valid. Note that as the θ parameter is increased the multipole approximation is accepted at smaller distances or for larger source node lengths, this tends to make the solution less accurate as the approximation usually becomes less valid for nodes that are larger or closer to the target.

Finally this results in several new arrays and functions to balance this Locally Essential Tree. We also represent a LET with a Cornerstone array and construction remains similar with some tweaks. First we compute whether the multipole approximation is accepted for every node outside of F, the multipole approximation is marked as not acceptable if it fails for any target node in F, the array of evaluations is called M. Some MACs may require communication between ranks. Rebalancing is now additionally a function of M and does not require communication while generating the histogram of node counts requires communication. The bottleneck according to Keller et al. [7] is the function that calculates the node properties based on particles and the node's children which is done via an upward pass using point-to-point communication for particles outside of F but where the LET has finer resolution than the global tree and collective communication for particles outside of F in nodes where the resolution of the global tree exceeds that of the LET; note that the LET has a variable resolution outside of F based on the MAC. The overall process is as follows then:

1. First the node properties for leaves in F are computed locally. Each rank finds nodes for point-to-point communication by traversing the global octree and evaluating as stated above.
2. Ranks send point-to-point messages of their octree nodes to peer ranks. The receiver uses the nodes to fill in properties for those halo nodes and then the node properties are sent back to the requesting peer.
3. Perform an *MPI_Allgatherv* communication to all nodes to send the aggregated LET properties across the ranks, then perform another upward pass on each node to get the properties all up to the global root node.

This yields a focused LET at each rank allowing traversal of the local subdomain and the calculation of forces.

3 Performance Methodology

The goal of this project is an analysis of Cornerstone [7] across varying parameter spaces, how the problem maps onto GPU hardware, in the distributed case the communication bottlenecks imposed by the algorithm, and to take advantage of any optimization opportunities that arise. There may be multiple opportunities for improvement as Cornerstone [7] appears to be mostly optimized for the distributed case. Much of the already existing performance analysis in regards to Cornerstone has been for extremely large particle counts with computation distributed across thousands of GPUs. For smaller application spaces such as real-time rendering or simulations on orders of magnitude around or less than 10^8, a single GPU or a few is likely more optimal due to the constraints imposed by communication costs, against the processing power of the a single GPU, as well as the significant reduction in work that the local case allows. A modern flagship GPU often has at least $40GB$ of DRAM, assuming a particle needs 32 bytes of storage we can fit on the order of 1 billion particles into the GPU. As we will see the overhead for distributing work is not trivial. Due to technical constraints, our preliminary results and analysis operate using only a single node with with 1 or 2 peer ranks each with their own GPU. However these type of workloads involving only modest numbers of GPUs are not uncommon especially in applications like computer aided engineering, and analysis on these workloads may yield useful conclusions for a number of use cases.

3.1 Code

To evaluate the Cornerstone octree construction method under realistic conditions, we developed a configurable batch generation pipeline that sweeps over a matrix of particle distributions, particle counts, and perturbation magnitudes. This hooks into our testing harness that runs the simulation with the ability to select between using GPUs or CPUs, the local only or distributed MPI case using a global tree and focus tree, and other parameters of interest. This is flexible enough to allow us to generate any metrics of interest in the future and has already allowed us to gather the results discussed in 4.

We created the necessary Python scripts to generate the initial particle distributions and the perturbed versions of the distribution used throughout this paper. This includes saving those distributions to HDF5 files, which could then be loaded in the C++ parts of the codebase to generate their corresponding octrees whether it be initial construction or a rebalance to an already existing octree. This allowed us to test many different types of distributions and their corresponding perturbation if they had one. Tooling to instrument sections of interest is done using NVIDIA Tooling Extensions (NVTX). We instrumented our code to highlight important metrics at a relatively coarse level so that we could observe the

primary steps necessary to building the Octree and focus our attention on bottlenecks. The program "perf" was also used to generate profiles on the CPU, which were also used to identify bottlenecks.

3.2 Distribution Generators

All initial particle distributions are confined to the cube domain $[-1, 1]^3$, where all distributions are as large as the domain. Six initial particle distributions are used:

1. **`uniform_initial`**: Samples particles uniformly at random from the domain.
2. **`pancake_initial`**: A cylindrical distribution along the Z-axis where the Z-axis is strongly compressed, as to resemble a disk, frisbee, or (as the name suggests) pancake.
3. **`pancake_initial_tilted`**: Same as the pancake but tilted off-axis, so the thin dimension is no longer aligned with a coordinate plane.
4. **`filament_z_initial`**: A one-dimensional filament aligned along the z-axis, characterized by the parametric equation: (0, 0, t).
5. **`filament_yz_initial`**: A filament lying in the yz-plane, anisotropic in two directions, characterized by the parametric equation: (0, t, t).
6. **`filament_xyz_initial`**: A filament oriented along the domain diagonal, anisotropic in all three coordinate directions simultaneously, characterized by the parametric equation: (t, t, t).

By analyzing 1D, 2D, and 3D manifolds that are or are not axis-aligned, a wide variety of behavior can be observed after applying perturbations. Because the perturbation distribution is 3-dimensional, the various distributions will have their unique attributes degraded as the perturbation magnitude increases.

3.3 Perturbation Model

After the initial distribution is generated, a perturbation vector is applied to simulate particle motion between timesteps. The perturbation function draws each displacement component independently from $\mathcal{U}(-s, +s)$ where s is the scale factor. Particles that move outside the domain after perturbation are truncated to the boundary (`out_of_bounds = 'truncate'`). Four perturbation scales are used:

- $s = 0.001$: Very small perturbation; particles barely move relative to the domain.
- $s = 0.01$: Moderate perturbation; noticeable displacement but locally coherent.
- $s = 0.1$: Large perturbation; significant displacement triggering substantial tree restructuring.
- $s = 0.5$: Very large perturbation; particles may traverse a large fraction of the domain.

3.4 Runtime Configuration

The Cornerstone framework accepts several runtime parameters via the command line:

- `-bucket-size-global`: Maximum number of particles per leaf node in the global octree (default: 1024).
- `-bucket-size-focus`: Maximum number of particles per leaf node in the focused octree (default: 64).
- `-theta`: The multipole acceptance criterion (MAC) parameter controlling the accuracy/performance tradeoff (default: 0.6).
- -gpu: Flag to enable GPU-accelerated octree construction.
- `-lets`: Flag to enable Cornerstone [7] to use Locally Essential Trees and communicate with peer ranks rather than use only local trees.

3.5 Test Configurations

The full sweep comprises 6 distributions × 5 particle counts × 4 perturbation scales = 120 configurations. There were also several tests run independently of this parameter sweep to gather general metrics which used a uniform distribution of particles on the unit cube.

3.6 Machines

For initial results on we used CloudLab to gain access to GPUs allowing us to get on a node with 4 V100s; For all the results presented in 4 we used a c4130 node on the CloudLab Wisconsin cluster. The node had four NVIDIA Tesla V100 GPUs. Each GPU had 16GB of memory in an SXM2 configuration with 900 GB/sec of peak memory bandwidth. A single V100 has 84 Streaming multiprocessors each with 64 FP32 cores, 64 INT32 cores, and 32 FP64 cores.

4 Results

In regards to local constructions, we first explored the performance breakdown between Morton and Hilbert keys. We used this breakdown to apply an optimization to the bottleneck of construction, and profiled the efficacy of such optimization. We also explored the execution time of an initial build versus the subsequent perturbation rebalance for qualitatively different initial distributions.

We compared the runtime between local construction and distributed construction for a low number of GPUs to assess the tradeoffs between the two methods. The memory usage was also profiled to assess how different factors such as key type and number of GPUs affected it.

4.1 Bottleneck

A general test to develop a sense of Cornerstone's internal bottlenecks is done on a single GPU comparing a case with 100 million particles using either Hilbert or Morton keys. The test loop contains the code to compute the SFC, do a radix sort, reorder the corresponding particle properties, and

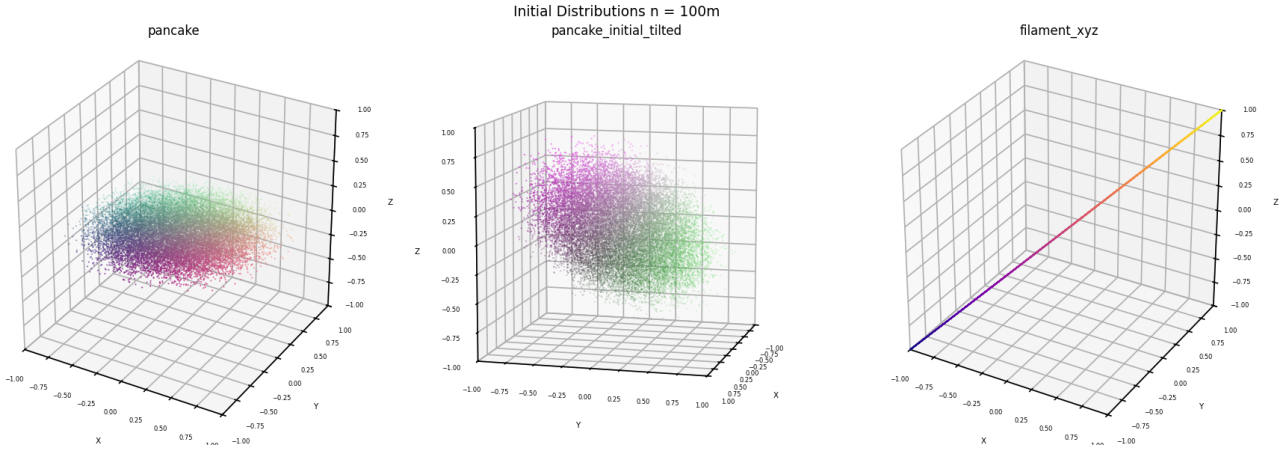

Figure 3. Initial particle distributions with 10^8 particles.

update the leaves and then internal tree state; these are the steps needed to compute the octree and have it in a usable state. Immediately from figure 4 we see that the sorting and reordering of particle properties is very dominant in this regime, with the computing of the SFCs taking only ≈ 10% of the total build time.

This is to be expected, as mentioned in 2, computation of SFCs for particles and the subsequent sorting and reordering operates on the total number of particles which is typically much greater than the number of nodes in the octree assuming N_{crit} is greater than 1, so the first part of the build has more work to do. This aligns with results from Keller et al. [7] when they profile Cornerstone. Their A100 profile is given below. The benchmark from their paper matches our breakdown here with Radix sort dominating although they do not consider the time to do gathers of particle properties to match the new layout.

Table 2. Original Cornerstone profile on an A100 GPU, adapted from [7].

execution on A100	1M	32M	512M
64-bit Morton keys	0.018 ms	0.31 ms	7.9 ms
64-bit Hilbert keys	0.084 ms	1.9 ms	30 ms
Radix sort	0.483 ms	5.0 ms	73 ms
Leaf Update	0.11 ms	0.51 ms	2.5 ms
Internal Nodes	0.19 ms	0.65 ms	5.4 ms

4.2 Vector Datatypes for Kernel Fusion

The bodies of which the octree is composed have associated properties. They will of course always have x, y, z, but they may have more such as mass, charge, and others. The particle SFC keys are sorted during octree construction, which leaves this information inaccessible when traversing the tree. Reordering the properties solves this problem using gather

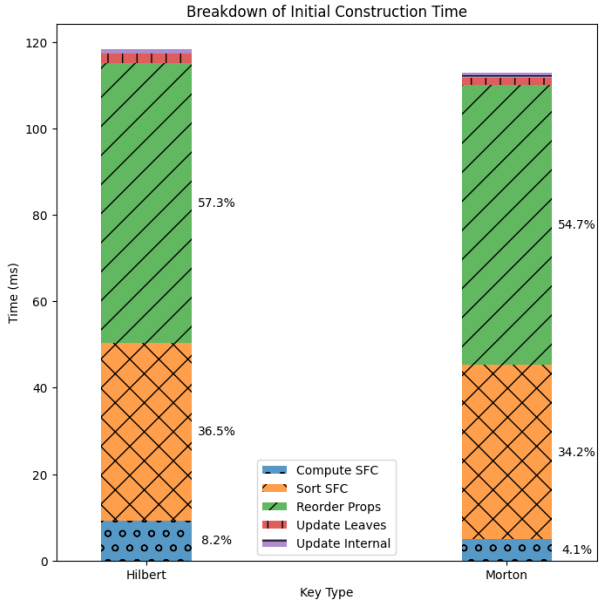

Figure 4. Comparison of operations in a broader tree update on a single GPU.

operations on the sorted indexes. These gathers and the sort are by far the most significant bottleneck in local octree construction.

We achieved significant optimizations in the case of local construction by utilizing vector datatypes and fusing the sort operation with the gathers to sort the vector data. One difference between CPU and GPU programming is that often times it makes more sense to utilize structures of arrays rather than arrays of structures. For example to represent particles in memory on the CPU one might create a struct in C which holds the properties and then create an array of these structures. This makes sense on the CPU because typically a CPU program will access other attributes of the

same element after accessing one and this makes for efficient caching. However on a GPU which implements a SIMT model of programming multiple threads in a warp execute the same instructions at the same time. In order to efficiently access global memory with a minimal number of transactions subsequent threads within a warp need to access memory that is contiguous. Typically a structure of arrays makes more sense in this case as the different threads will access the same attribute during the same cycle of the warp. For example if we instead create a particle structure which instead of representing a single particle has arrays for each of its properties an array for x, y, z then when multiple threads access subsequent elements of x they access contiguous memory which then coalesces to a $128 - bit$ load or store. For this reason Cornerstone uses arrays to represent the individual particle properties.

Another way to improve bandwidth is to use vector datatypes which allow one to group the properties but still load them efficiently. We utilized this structure instead which allowed us to sort all of the particle properties all at once rather than repeatedly issuing gathers. This ultimately resulted in up to 2.45× improvements in the overall build time of the octree as seen in figure 5.

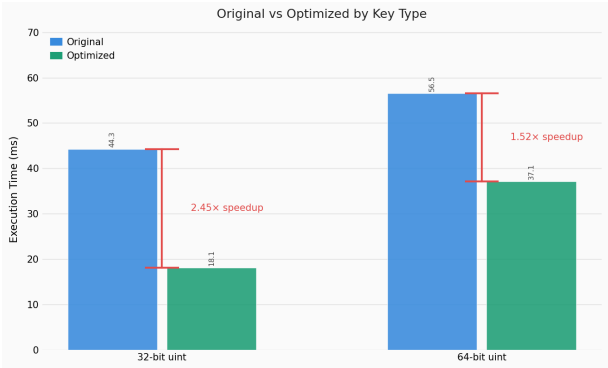

Figure 5. Performance improvements of optimized sort using vector datatypes over original gather for 100m particles on an A100 GPU.

4.3 Initial Distributions

For assessing the overall runtime of initial construction on qualitatively different distributions, there is a stark distinction of behavior between low particle count (1 million and less) and high particle count (10 million and more) workloads. The runtime of low particle workloads either is not correlated to the initial distribution, with each particle count's runtime distribution varying significantly. However, a consistent pattern emerges across high particle workloads, which is shown in Figure 7. Here we can see that, generally, lower dimension distributions have lower construction cost associated with them.

There is no consistently worst or best initial distribution as can be gleaned from figures 6, and 7. This is somewhat surprising, the author of Cornerstone, Keller mentioned that filaments and pancakes were difficult distributions for Cornerstone to process efficiently due to the large amounts of space that may be wasted in dividing the octants.

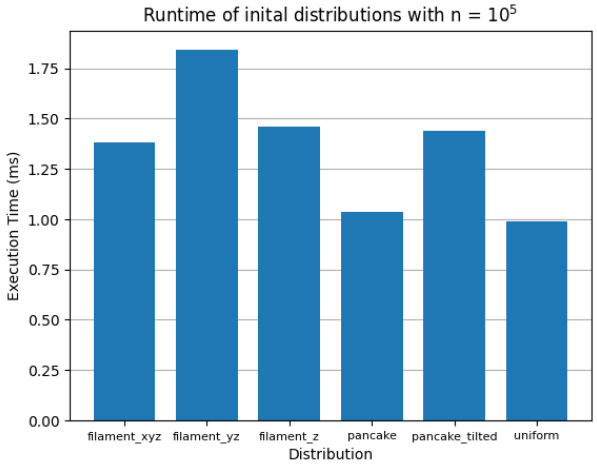

Figure 6. Construction time of different initial distributions for a low particle workload of n = 10^5.

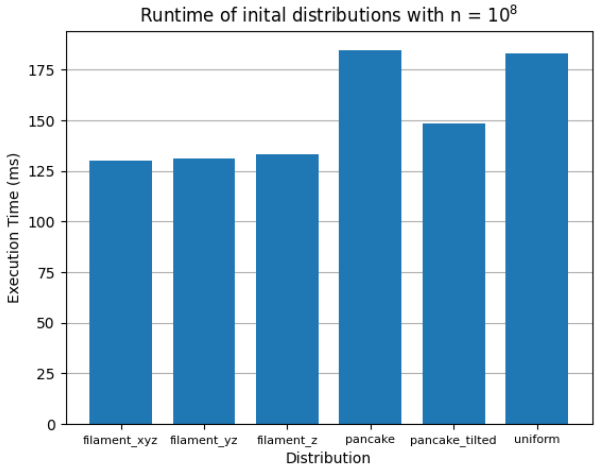

Figure 7. Construction time of different initial distributions for a high particle workload of n = 10^8.

4.4 Perturbations

To analyze the effects of changes to the particle distributions, we compared the update time of the octree after a perturbation is applied to the initial construction time for each initial distribution. It is clear that for the low particle workloads, perturbations have very little impact on update time as shown in figure 8. Only for high particle workloads does

the correlation of perturbation strength to relative update speed become significant, as shown in Figure 9.

The perturbations have varying effectiveness depending on the initial distribution to which they are applied. This is likely because the qualities that made the initial distributions quick to compute degrade as perturbation strength increases, and the distribution shifts closer to the uniform distribution. This is bolstered by the fact that the distributions which are most susceptible to perturbations are also the ones that have the lowest initial construction time. This is likely why the uniform distribution is not affected greatly by perturbations until a perturbation level of .5, when the filament distributions see a steady performance decrease at increasing perturbation levels. It is important to note that a relative execution time greater than 100% does not mean updating the octree is slower than reconstructing it from scratch as the perturbed distribution may still take longer to generate from scratch.

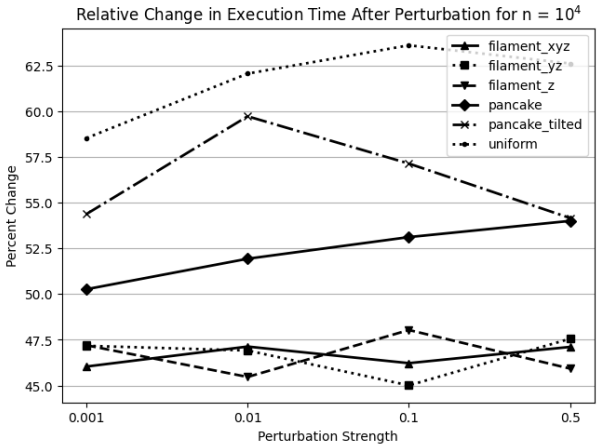

Figure 8. Perturbed octree rebalancing time relative to initial build time for different perturbation strengths for a low particle workload of n = 10^4.

4.5 Performance of Local Build vs. GPU Distributed

An important aspect of Cornerstone is that it supports distributed processing via domain decomposition and locally essential trees as mentioned in 2. We are interested in the overheads associated with building octrees which are partitioned across disaggregated memory modules. This has the benefit of being able to scale out and enables multiple GPUs to work on very large problems that could never fit into a single GPUs memory at once. One difference of note between our work and the problem that Cornerstone [7] is designed to solve is scale, Cornerstone's original purpose is smoothed particle hydrodynamics on exa-scale supercomputers, we are working on a single node on 1-4 GPUs. This difference in scale is likely to expose scalability issues different from

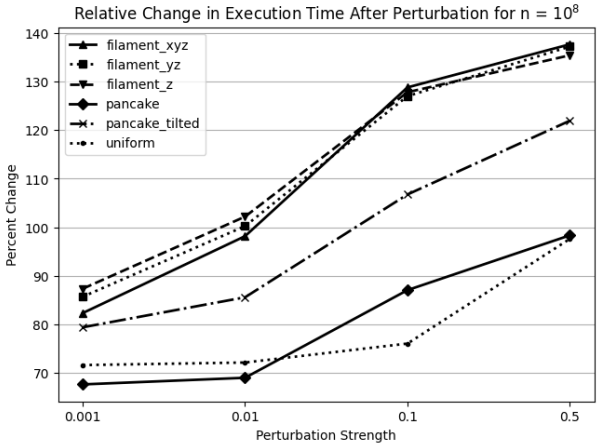

Figure 9. Perturbed octree rebalancing time relative to initial build time for different perturbation strengths for a high particle workload of n = 10^8.

the ones the authors needed to consider within the context of their work.

Similarly to the local case, we profiled the distributed method to gauge where bottlenecks were. A flamegraph revealed that a majority of the computation time was spent finding peer ranks, which is a CPU side computation. What is notable is that the communication cost was negligible. This is likely because of the small amount of GPUs tested on because NVLINK was able to greatly lower communication overhead.

We observed the weak scaling of Cornerstone in figure 10 for every combination of configuration options for numbers of particles between 1 million and 100 million; the configuration options included the space filling curve type being either Hilbert or Morton, the number of bits per key being either 32-bits or 64-bits, and the number of GPUs used being either the single GPU local case, or decomposed using 2 or 4 GPUs. In every case the local construction was between 1-2 orders of magnitude faster; this difference in performance is large enough that we can conclude that there is no performance advantage of using distributed construction over local when it is possible to fit the particles on a single GPU. Note that we profile up to the 100 million particles mark, if we were to use the GPU DRAM very well and get within some small factor of filling DRAM we could fit at best, $\frac{TotalDRAM}{sizeof(P)}$ as the number of particles a GPU can support. In the case of an A100 with $40GB$ of memory and the assumption that a particle consumes 32 bytes of memory tells us we would be able to fit at most $1,342,177,280$ particles. Such a model is far too optimistic as we will see but this serves to show that even with a very optimistic memory model we are within a factor of 10 of the maximum number of particles a GPU can support which should serve to generalize our conclusion to that there is no performance benefit to using distributed if the number of

particles is in this domain. As we can see from figure 10 even with 32 bit Morton keys the local case still beats distributed by roughly an order of magnitude. Given this conclusion the question that drives performance in this regime is at what point the GPUs memory is exhausted and how well we can scale the computation out given the overheads implied by using decomposition.

We explore the scalability of Cornerstone with respect to memory in figure 11. The picture painted by this figure is more concerning, what is clear is that the memory overhead Cornerstone imposes is large enough to be very prohibitive. Given a fixed number of particles on a GPU one might hope that by doubling the number of GPUs, the memory consumed per GPU would roughly split in half since half of the particles would be on the other GPUs. This is clearly not the case, moving from 2 to 4 GPUs gives a much smaller decrease typically between 10%−20%. This has significant implications because to increase the number of particles we require more GPUs to fit them which aside from the performance impacts of needing to use more GPUs one also needs to consider the financial impact of this result. With the total memory being reduced at a rate of roughly 10% − 20% given double the number of GPUs the cost borne to increase capacity is restrictive.

Digging in to figure 11 further we see the source of the problem is the overhead of Cornerstone, even though the memory of particles is of course being halved by doubling the number of GPUs, the overhead of Cornerstone's internal structures seems to remain quite fixed. Key types had little to no impact on the memory usage but key sizes did noticeably lower memory usage in all cases. It is important to note that there is no data for local construction on 500 million particles as one GPU ran out of memory, with 200 million being the largest particle count we found that could still be constructed locally.

5 Future Work

Of particular interest for future work is the scalability of Cornerstone regarding memory. As mentioned in 4.5 the factor by which memory decreases due to a doubling in GPU hardware is not close to 50% as it would be in a totally ideal case, it was typically between 10% − 20% at the number of particles we tested. In order to efficiently reduce the memory consumption, which as we showed becomes the bottleneck, one would need to determine which structures within Cornerstone are causing scalability issues and optimize them where ever possible with an ideal although unrealistic target of reaching a 50% reduction in particles when doubling the number of GPUs.

The method presented in 4.2 provides superior performance; however, it would need extensions to be more general and work for an arbitrary number of particle properties. Currently because the Radix sort is implemented via CUB the source code is unavailable, however, one could build a custom Radix Sort which sorts all properties within the kernel, achieving the same kernel fusion effect but with a pack of arrays rather than a single array using NVIDIA vector datatypes. This would be more generalizable than the vector datatype implementation because it can handle such arbitrary numbers of properties. NVIDIA also reportedly has plans to allow the CUB Radix sort to do byte-wise reordering which would also solve this problem. Additionally, a Thrust version could be implemented using zip iterators.

Octree's are structures built to service an algorithm whether it be collision detection, force calculations via Barnes Hut or Fast Multipole Method, graphics, or other applications. Understanding in what application contexts and under what conditions the tree build time is a bottleneck informs whether or not it is worth having a very optimized implementation of tree builds. Cornerstone in particular has a unique feature that effects performance being that the tree is rebalanced rather than having to rebuild this may be advantageous in some applications but cause unneeded complications in other problems.

6 Related Work

Karras [6] developed a fully parallel binary radix tree construction method for the GPU. Cornerstone builds directly on these algorithms, extending them from single-GPU graphics to distributed with locally essential trees (LETs) and iterative rebalancing. Apetrei [2] improved on Karras by fusing bounding box calculation into a single bottom-up kernel pass. Cornerstone also minimizes kernel launches but operates on the leaf-level SFC key array rather than an explicit node hierarchy, helping accelerate rebalancing.

Warren and Salmon [10] introduced the parallel hashed octree, replacing pointer-based structures with hash tables for distributed N-body simulation. Cornerstone shares the distributed octree goal but replaces the hash scheme with a linear SFC-key representation that maps naturally onto GPU scan kernels.

Burtscher and Pingali [4] presented the first complete GPU-only Barnes-Hut implementation, showing 74x speedup over optimized serial CPU code. Unlike Cornerstone, their tree is rebuilt from scratch each timestep and targets only the single-GPU case. Bédorf et al. [3] developed Bonsai, a GPU-resident sparse octree treecode processing over 2.8M particles/sec with 20x speedup over CPU. Cornerstone follows the same GPU-resident philosophy but adds distributed multi-GPU support and rebalancing rather than full reconstruction.

Sundar et al. [9] presented parallel bottom-up construction and 2:1 balance refinement of linear octrees, handling over a billion octants in under a minute on 1024 CPUs. While both DENDRO and Cornerstone target scalable octree construction, DENDRO is designed for CPU-based adaptive mesh

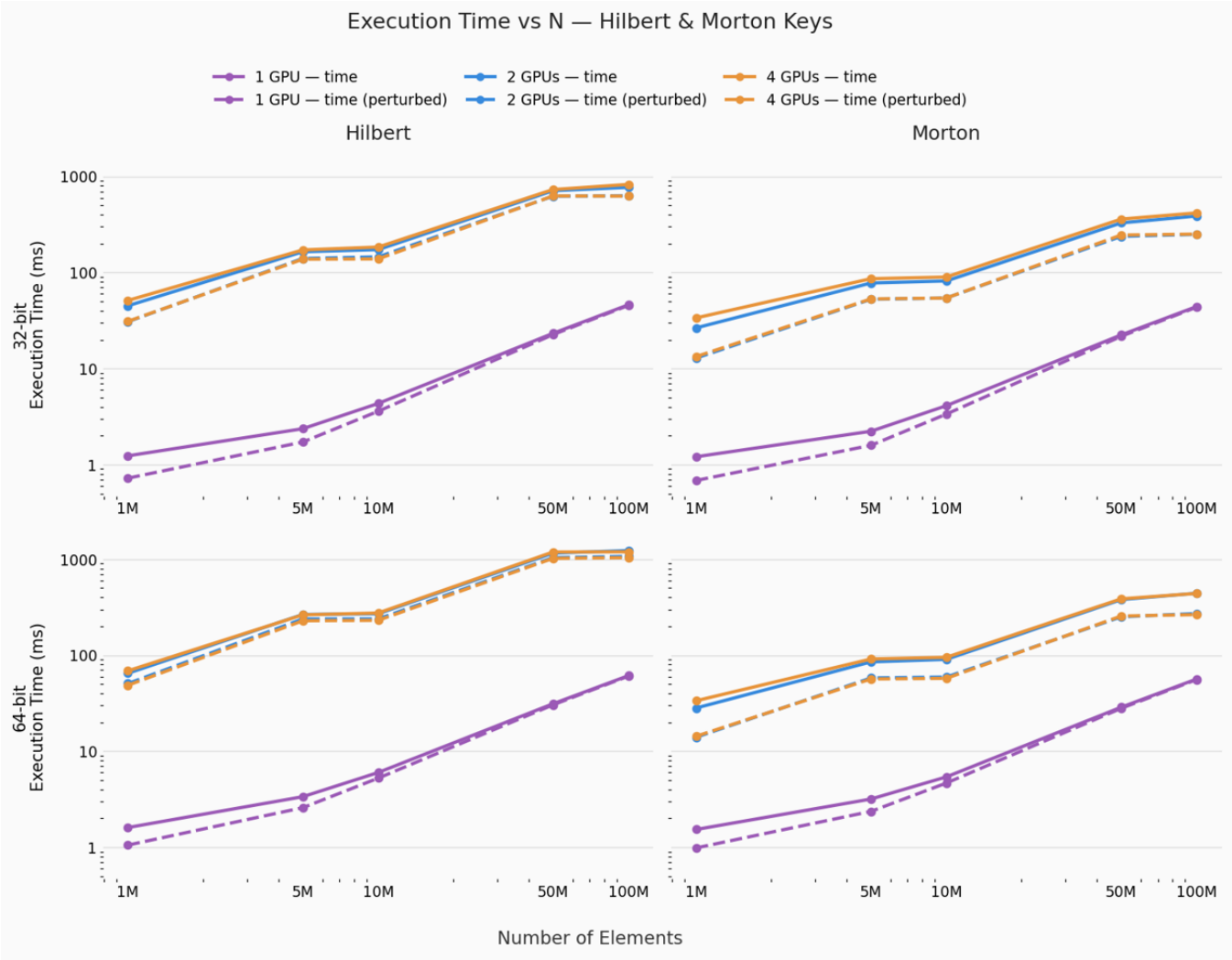

Figure 10. Performance of different numbers of GPUs across different key configurations on an A100 node.

PDE solvers, while Cornerstone targets GPU particle simulations with incremental updating.

Lauterbach et al. [8] compared Morton-code LBVH builds against SAH approaches for GPU BVH construction, noting the construction speed versus traversal quality tradeoff. Cornerstone uses a similar Morton encoding but targets particle simulations and allows Hilbert curve usage. Deng et al. [5] introduced the G-ML-Octree, showing an order of magnitude update cost reduction for moving 3D objects on GPUs through a loose octree with memoized updates. Both G-ML-Octree and Cornerstone avoid full reconstruction, but Cornerstone achieves this by iteratively adjusting the SFC key boundaries of its leaf array instead of maintaining a loose hierarchy with memoized updates.

Adinets and Merrill [1] developed the current standard in GPU Radix Sorts called Onesweep which provided significant improvements over prior Radix Sort implementations. As discussed in 5 one potential improvement of our work is to implement a custom Radix Sort which merges the sort with the gathers of particle properties.

7 Conclusion

In summary, we have presented our initial efforts towards analyzing Cornerstone [7]. The Cornerstone [7] code and ours were instrumented using the NVTX profiling tools. We developed a harness for running the application using GPUs or CPUs and with either a local octree setup or a distributed setup. Around this we created scripts to run parameter sweeps and gather profiles and the octree view. We also developed scripts to generate the distributions of initial positions and their perturbations. And we detailed the kernels and code that are required by the algorithm.

To gather initial results we used a CloudLab node with 4 V100 GPUs. We profiled the local octree case and a case with

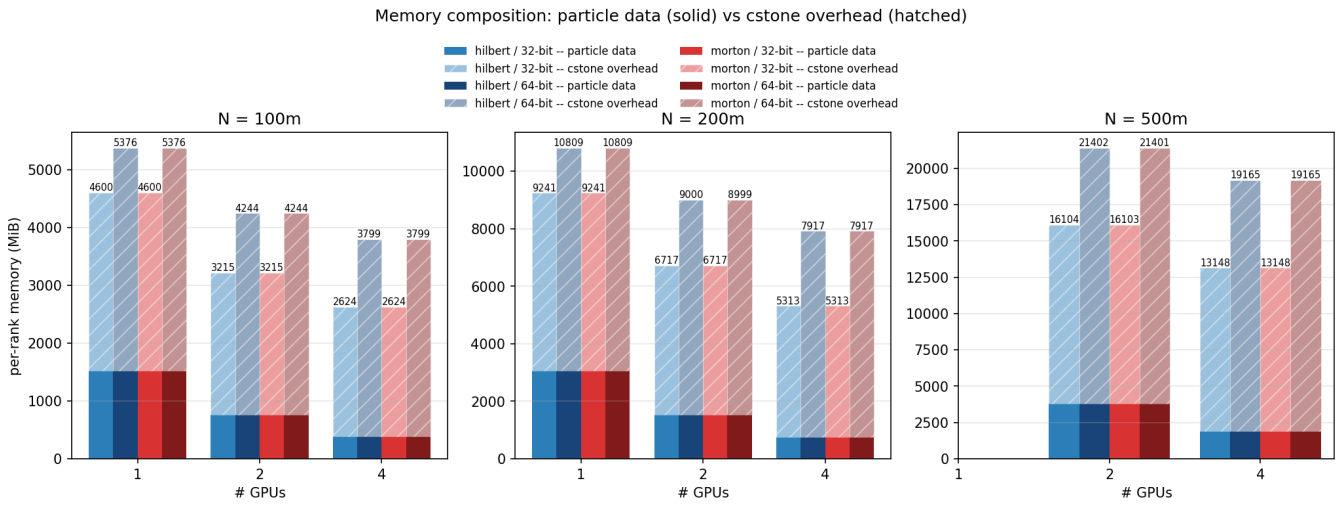

Figure 11. A profile of the memory usage in MiB for varying numbers of GPUs, problem sizes, key bits and types. The chart is divided into a solid region which shows the portion of memory dependent on the particle data and the hatched region which shows the overhead unrelated to the memory consumed by particles specifically; this includes Cornerstone's internal data structures.

2 MPI ranks each with their own V100 GPU. We showed that perturbation strength did correlate with the rebalancing time but only with larger numbers of particles. The sorting and reordering steps dominate the computation of the tree locally. We also showed that the distributed case introduced significant overhead near two orders of magnitude but was not due to communication costs which needs further exploration.

8 Metadata

The code/data of the project can be found at:

https://github.com/Murdock022X/pca533-octrees

References

[1] Adinets, A., and Merrill, D. Onesweep: A faster least significant digit radix sort for gpus, 2022.

[2] Apetrei, C. Fast and Simple Agglomerative LBVH Construction. In *Computer Graphics and Visual Computing (CGVC)* (2014), R. Borgo and W. Tang, Eds., The Eurographics Association.

[3] Bédorf, J., Gaburov, E., and Portegies Zwart, S. A sparse octree gravitational n-body code that runs entirely on the GPU processor. *Journal of Computational Physics 231*, 7 (2012), 2825–2839.

[4] Burtscher, M., and Pingali, K. An efficient CUDA implementation of the tree-based Barnes Hut n-body algorithm. In *GPU Computing Gems Emerald Edition*. Morgan Kaufmann, 2011, ch. 6, pp. 75–92.

[5] Deng, Z., Wang, L., Han, W., Ranjan, R., and Zomaya, A. Y. G-ML-Octree: An update-efficient index structure for simulating 3D moving objects across GPUs. *IEEE Transactions on Parallel and Distributed Systems 29*, 5 (2018), 1075–1088.

[6] Karras, T. Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees. In *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics* (2012), C. Dachsbacher, J. Munkberg, and J. Pantaleoni, Eds., The Eurographics Association.

[7] Keller, S., Cavelan, A., Cabezon, R., Mayer, L., and Ciorba, F. Cornerstone: Octree construction algorithms for scalable particle simulations. In *Proceedings of the Platform for Advanced Scientific Computing Conference* (New York, NY, USA, 2023), PASC '23, Association for Computing Machinery.

[8] Lauterbach, C., Garland, M., Sengupta, S., Luebke, D., and Manocha, D. Fast BVH construction on GPUs. *Computer Graphics Forum 28*, 2 (2009), 375–384.

[9] Sundar, H., Sampath, R. S., and Biros, G. Bottom-up construction and 2:1 balance refinement of linear octrees in parallel. *SIAM Journal on Scientific Computing 30*, 5 (2008), 2675–2708.

[10] Warren, M. S., and Salmon, J. K. A parallel hashed oct-tree n-body algorithm. In *Proceedings of Supercomputing '93* (1993), ACM, pp. 12–21.